

Lecture # 02

Programming with Structures

OBJECT ORIENTED PROGRAMMING (OOP)

- Most dramatic innovation in software development in the last decade
- Before OOP, the prevalent programming paradigm was Structured Programming
- Structured Programming
 - A large program involving complex logic is divided into control structures and functions
 - Result: more easily maintained code

STRUCTURED PROGRAMMING

- **Problems with Structured Programming**
 - Cannot cope with very large project sizes
 - Expensive software errors
(e.g. air traffic control)
- **Goal of OOP**
 - Clearer, more reliable, more easily maintained programs
 - More effective way of coping with program complexity

STRUCTURED PROGRAMMING

- **Causes of Problems with Structured Programming (using Procedural Languages)**
 - Unrestricted Access to Data
 - Access to data by multiple functions leads to inconsistencies, and side-effects
 - Programs become difficult to modify and maintain

STRUCTURED PROGRAMMING

- Poor Modeling of Real World Things
 - Real world things are integral collections of data and functions
 - e.g. a car: has data (make, model etc.) and functions (acceleration)
 - Procedural languages do not tie up data with
- **Difficulties in Creating New Data**
 - e.g. complex nos., date, rational nos. etc.
 - Language Extensibility

THE OO APPROACH (CHARACTERISTIC OF OO LANGUAGES)

- The fundamental idea is to combine data and functions into a single unit that operate on the data.
- Such a unit is called an “Object”.
- An object’s functions are called “member functions” in C++
- And its data is called “data members”.

OBJECT ORIENTED PROGRAMMING

CLASSES

- Objects belong to classes
- All objects with the same characteristics (data and functions) constitute one class.
- A class serves only as a plan, or a template, or sketch.
- It merely specifies what data and what functions will be included in objects of that class.

CLASSES

- Declaring a class doesn't create any objects, just as mere existence of data type `int` doesn't create any variables.
- A class is thus a description of a no. of similar objects.
- For instance, `HUMAN` is a class, and `OMER` is its instance (object)

INHERITANCE

- Derive other classes from an existing class.
- Each class shares common characteristics with the class from which it was derived, and can also add its own modifications, additions.
- For instance, VEHICLE is a class from which CAR, TRUCK, BUS, MOTORCYCLE classes can be derived.

INHERITANCE

- The original class is called the BASE CLASS; the others are DERIVED CLASSES
- Re-usability
- Language extensibility (create new data types)

POLYMORPHISM AND

OVERLOADING

- Using operators or functions in different ways depending on what they are operating on is called polymorphism
- Overloading is a special case of polymorphism, e.g. +, -, /, << etc.

OVERLOADED FUNCTIONS

- An overloaded function performs different activities depending on the kind of data sent to it
- Different no. of arguments
- Different types of arguments
- Or both

```
#include <iostream>
using namespace std;
void repchar(); //repeats one character horizontally
void repchar(char);
void repchar(char, int);
int main()
{
    repchar(); repchar('='); repchar('+', 30);
    return 0;
}
void repchar()
{
    for (int j=0; j<45; j++)
        cout<<'*';

    cout<<endl;
}
```

```
void repchar(char ch)
{
    for (int j=0; j<45; j++)
        cout<<ch;
    cout<<endl;
}
void repchar(char ch, int n)
{
    for (int j=0; j<n; j++)
        cout<<ch;
    cout<<endl;
}
```

Structures

User Defined Data Types

A structure is a user-defined data type in C/C++. A structure creates a data type that can be used to group items of possibly different types into a single type.

For example: You want to store some information about a person: his/her name, citizenship number and salary. You can easily create different variables name, citNo, salary to store these information separately.

However, in the future, you would want to store information about multiple persons. Now, you'd need to create different variables for each information per person: name1, citNo1, salary1, name2, citNo2, salary2.

You can easily visualize how big and messy the code would look. Also, since no relation between the variables (information) would exist, it's going to be a daunting task.

A better approach will be to have a collection of all related information under a single name Person, and use it for every person. Now, the code looks much cleaner, readable and efficient as well.

This collection of all related information under a single name Person is a structure.

Data Structure

- A *data structure* is a group of data elements grouped together under one name.
- These data elements, known as *members*, can have different types and different lengths.

How to create a structure?

- The 'struct' keyword is used to create a structure. The general syntax to create a structure is as shown below:

```
struct structureName{  
    member1;  
    member2;  
    member3;    .  
    .    memberN;  
};
```

Data Structure

- Data structures can be declared in C++ using the following syntax:
- ```
struct type_name {
 member_type1 member_name1;
 member_type2 member_name2;
 member_type3 member_name3;

} object_names;
```

When a structure is created, no memory is allocated.

- The structure definition is only the blueprint for the creating of variables. You can imagine it as a datatype. When you define an integer as below:
- `Int foo;`

The `int` specifies that, variable "foo" can hold integer element only. Similarly, structure definition only specifies that, what property a structure variable holds when it is defined.

# Data Structure

- For example:

```
struct product {
 int weight;
 double price;
} ;
product apple;
product banana, melon;
```

# Data Structure

This declares a structure type, called product, and defines it having two members:

weight and price,

This declaration creates a new type (product), which is then used to declare three objects (variables) of this type:

apple, banana, and melon.

Note how once product is declared, it is used just like any other type.

# Data Structure

Right at the end of the struct definition, and before the ending semicolon (;), the optional field `object_names` can be used to directly declare objects of the structure type.

For example, the structure objects `apple`, `banana`, and `melon` can be declared at the moment the data structure type is defined:

# Data Structure

```
struct product {
 int weight;
 double price;
} apple, banana, melon;
```

It is important to clearly differentiate between what is the structure type name (product), and what is an object of this type (apple, banana, and melon).

Many objects (such as apple, banana, and melon) can be declared from a single structure type (product).

Once the three objects of a determined structure type are declared (apple, banana, and melon) its members can be accessed directly. The syntax for that is simply to insert a dot (.) between the object name and the member name. For example, we could operate with any of these elements as if they were standard variables of their respective types:

```
apple.weight
apple.price
banana.weight
banana.price
melon.weight
melon.price
```

# **Structures in C++ can contain two types of members:**

## **Data Member:**

- These members are normal C++ variables. We can create a structure with variables of different data types in C++.

## **Member Functions:**

- These members are normal C++ functions. Along with variables, we can also include functions inside a structure declaration.

```
// Data Members
```

```
int roll;
```

```
int age;
```

```
int marks;
```

```
// Member Functions
```

```
void printDetails()
```

```
{
```

```
 cout<<"Roll = "<<roll<<"\n";
```

```
 cout<<"Age = "<<age<<"\n";
```

```
 cout<<"Marks = "<<marks;
```

```
}
```

In this structure, the data members are three integer variables to store *roll number*, *age* and *marks* of any student and the member function is *printDetails()* which is printing all of the above details of any student.

C++ Program to assign data to members of a structure variable and display it.

```
1.#include <iostream>
2.using namespace std;
4.struct Person
5.{
6.char name[50];
7.int age;
8.float salary;
9.};
11.int main()
12.{
13.Person p1;
15.cout << "Enter Full name: ";
16.cin.get(p1.name, 50);
17.cout << "Enter age: ";
18.cin >> p1.age;
19.cout << "Enter salary: ";
20.cin >> p1.salary;
22.cout << "\nDisplaying Information." << endl;
23.cout << "Name: " << p1.name << endl;
24.cout << "Age: " << p1.age << endl;
25.cout << "Salary: " << p1.salary;
27.return 0;
28.}
```

## Output

Enter Full name: Magdalena  
Dankova

Enter age: 27

Enter salary: 1024.4

Displaying Information.

Name: Magdalena Dankova

Age: 27

Salary: 1024.4

Here in example a structure Person is declared which has three members name, age and salary.

Inside main() function, a structure variable p1 is defined. Then, the user is asked to enter information and data entered by user is displayed.

## **C++ Structure and Function**

Structure variables can be passed to a function and returned in a similar way as normal arguments.

```

1.#include <iostream>
2.using namespace std;
4.struct Person
5.{
6.char name[50];
7.int age;
8.float salary;
9.};
11.void displayData(Person); // Function declaration
13.int main()
14.{
15.Person p;
17.cout << "Enter Full name: ";
18.cin.get(p.name, 50);
19.cout << "Enter age: ";
20.cin >> p.age;
21.cout << "Enter salary: ";
22.cin >> p.salary;
24.// Function call with structure variable as an argument
25.displayData(p);
27.return 0;
28.}
30.void displayData(Person p)
31.{
32.cout << "\nDisplaying Information." << endl;
33.cout << "Name: " << p.name << endl;
34.cout << "Age: " << p.age << endl;
35.cout << "Salary: " << p.salary;
36.}

```

## Output

Enter Full name: Bill Jobs

Enter age: 55

Enter salary: 34233.4

Displaying Information.

Name: Bill Jobs

Age: 55

Salary: 34233.4

In this program, user is asked to enter the name, age and salary of a Person inside main() function.

Then, the structure variable p is passed to a function